

AMR rulebook

Every rule the robots obey, taken from the code rather than from memory. Each entry gives the rule, why it exists — usually a failure that was measured — and where it lives. Edit this list and the code should follow, not the other way round.

A Right of way — when two robots meet

- A1** The LOWER-numbered robot gives way. amr1 stands aside for everybody; amr2 for everybody except amr1, and so on up the fleet.
- Total and fixed, so two robots can never both think they have the road. Compared numerically, not as text — text order reads amr10 as lower than amr2.
- `who_yields(), _yield_rank()`
-
- A2** Who yields is decided ONCE per encounter and remembered until the encounter ends.
- An earlier rule recomputed it every tick from live positions and it flipped the moment either robot moved: both gave way, both rejoined, both met again, and they touched inside that loop.
- `SimAcs._giving_way`
-
- A3** A robot may be in only ONE encounter at a time. A third robot arriving is refused and queues instead.
- Pair-keyed encounters let three robots open three encounters in one second, each naming its own yielder. One robot became the yielder twice over with a single lay-by to satisfy both.
- `who_yields() – engagement check`
-
- A4** If the chosen yielder has no lay-by it can reach, the duty passes to the other robot ONCE. If both refuse, the decision stands and A9 ends it.
- Number order knows nothing about room. The other robot is by construction standing where this one wanted to be, so it has the space this one lacks.
- `SimAcs.cannot_yield()`
-
- A5** The passer does not move until the yielder reports the explicit all-clear. It does not judge by the gap.
- The passer used to start as soon as layer 1 stopped objecting, which is part way through the other robot's move aside. It drove into a robot that was still getting out of its way. One robot moves at a time.
- `_handle_give_way() – passer branch`
-
- A6** The yielder drives to its lay-by, stops, and HOLDS. It does not resume its own goal while the encounter is open.
- A give-way is a promise to stay put; the passer is holding on the strength of it. Returning control to the drive path made the yielder bounce in and out of the lane 29 times in 10 minutes.
- `_handle_give_way() – yield branch`
-
- A7** The road is clear only when NO oncoming robot is still ahead. If a robot follows the one being let past, the yielder keeps waiting for all of them.
- Rejoining the moment the partner is past puts the yielder back into the robot behind it, mid-turn, in front of a robot that had right of way.
- `_oncoming_ahead()`
-
- A8** An encounter is abandoned if the partner is more than 8.00 m away.
- Two robots that far apart are not in an encounter, whatever the bookkeeping says. Measured: a robot held for a partner thirteen metres away.
- `ENCOUNTER_RANGE = 8.0 m`
-

-
- A9** A robot that has given way for 45 s with nobody passing gives up, ends the encounter, and fails its job if it has one.
- The last resort. The clock starts before any early return, so a yielder that answers without moving is still bounded.
- `YIELD_LIMIT = 45.0 s`

B Overtaking and queueing

- B1** Robots travelling the SAME direction never overtake.
- A follower waits, or gives way itself if it must. It does not pass.
- `_robot_ahead(), _yielder_ahead()`
-
- B2** A robot that is giving way is STILL ON THE ROAD and may not be passed at any distance.
- It is holding a gap open for one named robot. Everybody else queues. There is no clearance to compute, because nobody squeezes past.
- `_yielder_ahead()`
-
- B3** Only a robot that has turned off onto a SPUR — docking, or parked — has left the road and may be driven past.
- Measured against the road graph, not a radius. The old 3.80 m bay circle reached the aisle, so a robot standing in the road answered 'you may pass' and two bodies touched.
- `_on_a_spur()`
-
- B4** A robot queues behind a yielder it can see within 8.00 m ahead, measured along its own travel.
- A yielder is off to the side and no longer on the lane line, so a test against the lane would never see it. What matters is whether it is in front.
- `QUEUE_LOOKAHEAD = 8.0 m`
-
- B5** The protective corridor is 2.40 m ahead and 1.20 m to each side, measured along the direction of travel, not where the nose points.
- The platform crabs, so heading and travel are different things. Uses fleet poses, not the scanners: a range reading cannot tell a robot from a machine.
- `ROBOT_STOP_AHEAD, ROBOT_STOP_SIDE`

C Collision avoidance — layer 1

- C1** No two robot bodies may come within 0.30 m. Checked by sampling both footprints forward 2.00 s at 0.25 s steps, at their measured velocities.
- The last word before any velocity is published. Past this line every path — driving, docking, reversing out, homing — has been checked.
- `STOP_GAP = 0.30 m, LOOKAHEAD_S = 2.0 s`
-
- C2** Layer 1 ignores a robot that is standing still.
- A constant separation never closes. This is what lets a robot drive past one parked in a spur — and it is why B2 exists as a separate rule.
- `_threat()`
-
- C3** Layer 1 only ever says STOP. It never decides who goes.
- Two robots frozen facing each other is the correct failure for this layer. Deciding between them is the job of the rules above it.
- `drive() – layer 1 block`
-

-
- C4** If layer 1 has been stopping a robot for the same other robot for 4 s and both are stationary, the fleet is asked to decide who yields.
Two robots crossing are neither head-on nor following, so nothing above layer 1 ever chose. Measured: two robots sat 2.02 m apart for two minutes.
`DEADLOCK_AFTER_S = 4.0 s`
-
- C5** Laser repulsion steers the robot but never halts it.
It nudges. It is deliberately bounded so it never becomes the thing that drives the robot; stopping is layer 1's job alone.
`_repulsion(), max_repulsion = 0.85`
-
- C6** A robot with no ground-truth pose never commands a wheel.
Never drive blind.
`drive() – first guard`
-

D Junctions

- D1** A junction is held by one robot at a time.
The red light where a spur meets an aisle.
`claim_junction()`
-
- D2** NO ROBOT EVER WAITS ON A JUNCTION WHILE HOLDING ONE. A robot refused a junction releases the one it holds.
This is what makes a circular wait impossible. Enforced centrally so every caller gets it rather than having to remember. Measured: two robots each sat on the junction the other needed until a job timed out.
`claim_junction() – failure path`
-
- D3** A robot that becomes a yielder releases its junction immediately.
Standing aside frees the road but used to keep the red light. If that was the junction the passer needed, the passer could never pass.
`who_yields() – release`
-
- D4** A robot driving home obeys junctions like any other robot.
Homing is an ordinary trip on the ordinary lanes. A homing robot that ignored the lights was an unreserved robot crossing reserved junctions, and it never released the one it arrived holding.
`_go_home()`
-

E Docking

- E1** One robot per station, ever. A second is refused entry and holds outside.
The protocol carries exactly one 'AGV is inside' bit per docking axis, so a second robot has nowhere to report itself even if it wanted to.
`request_entry()`
-
- E2** The robot squares up to the machine face BEFORE crabbing in.
At the approach point the face is 2.2 m away and the robot's half-diagonal is 0.918 m, so there is room to turn there. There is none once it has crabbed in.
`_square_up()`
-
- E3** It stops 0.65 m from the face, and no nearer than 0.48 m.
Half the robot's width plus a 0.20 m gap.
`DOCK_TARGET, DOCK_MIN`
-

-
- E4** Obstacle repulsion fades out over the last 2.20 m of the approach.
The approach point sits close to a solid machine. Without the fade the robot would refuse to dock at all.
`dock_fade_m = 2.2 m`
-
- E5** It dwells 3 s at the port to load or unload.
A dwell is not a stall, so it is checked before the stall watchdog.
`dwell_seconds = 3.0 s`
-
- E6** It must clear the bay within 8 s of finishing, or it gives up and releases the interlock.
It still holds the interlock while reversing out, and the next robot is waiting on it.
`_exit_stalled()`
-

F Battery

- F1** An idle robot below 30 % is sent to charge.
Only when idle. A job in progress is not abandoned for a charge.
`LOW_BATTERY = 30.0 %`
-
- F2** It charges to 90 %.
`CHARGE_TO = 90.0 %`
-
- F3** Below 12 % it goes to charge even if that means abandoning work.
The point past which finishing the job risks stopping in an aisle.
`CRITICAL_BATTERY = 12.0 %`
-
- F4** At 0 % the robot stops where it stands and is not recovered automatically.
Exactly as on a real floor: something has to come and get it. What the CSM must do is never let it happen.
`drive() – battery guard`
-
- F5** A robot whose control chain is down is given no work.
A pose exists the moment a model is spawned, before its controllers — so pose cannot answer this. `joint_states` can: it stops when the chain dies.
`can_move()`
-

G Homing and parking

- G1** A robot with no work returns to its parking slot.
A robot that simply stops where it finished is a road block.
`_go_home()`
-
- G2** Each robot has one slot, fixed by its leg and its number.
It has to be stable, because a robot drives home to it. A robot's slot does not move when the fleet grows.
`plant.PARKING_SLOTS`
-
- G3** Every second slot has a charger. A charger is a parking slot with a cable, not separate geometry.
That is how the real floor works — a robot waiting and a robot charging are in the same row.
`plant.CHARGERS`
-
- G4** A robot may use only its OWN leg's chargers, nearest first, and must claim one before setting off.
Driving across the plant to another leg's charger would cross every lane it is meant to stay out of. Preference is not reservation.
`chargers_for(), claim_charger()`
-

H Job assignment

- H1** A robot serves ONE leg of the material flow. One dictionary entry binds it.
Everything else — roads, markers, parking, docking, the job FSM — derives from that entry. Adding a robot is a data change, not code.
`plant.ROBOT_SEGMENT`
- H2** A job for a leg whose robots are all busy is answered BUSY, not REJECTED. It waits its turn.
The job is perfectly valid; the robot class that serves it is working. This is what lets a leg have no robot at all.
`_dispatch()`
- H3** Two robots are never sent to the same DESTINATION. Sources are deliberately not claimed.
There is nothing to arbitrate at a drop-off. Claiming sources would serialise three machines fed by one store and leave two robots idle; the shared pickup is arbitrated by the entry interlock instead.
`_dispatch() – taken set`
- H4** Among free robots on the right leg, the one nearest the pickup is chosen.
The only decision the ACS really owns.
`_dispatch() – distance_to_pickup`

I Failure bounds — every wait is finite

- I1** A robot that moves less than 0.12 m in 8 s while driving fails the job as PATH BLOCKED.
Ground truth, not wheel odometry: odometry reports progress happily while the chassis is wedged against a pallet.
`stall_seconds = 8.0 s, stall_distance = 0.12 m`
- I2** Giving way is bounded at 45 s.
See A9.
`YIELD_LIMIT`
- I3** A job is abandoned after 600 s.
`job_timeout = 600 s`
- I4** Clearing a bay is bounded at 8 s.
See E6.
`_exit_stalled()`

J The order the rules run in

Each tick, every robot is put through these in order. The first one that applies consumes the tick — so a rule can only be reached if every rule above it let the robot through.

1	Ground truth No pose, no movement.
2	Battery Flat means stopped, whatever else is true.
3	Right of way The give-way handshake, applied to EVERY robot — idle, homing, reversing out or carrying a roll.
4	Layer 1 The last word before any velocity is published.

5	Queue behind a yielder Right of way, not collision — so it sits after layer 1.
6	Clear the bay Nothing else happens until the interlock is released.
7	Go home Only if there is no work.
8	Do the job Drive, dock, dwell, deliver.

K Known gaps — rules that are NOT enforced

- K1** A robot may claim a junction it cannot drive through. `can_move()` gates job dispatch but not junction claiming, so a robot whose controllers die holds that junction indefinitely.
- K2** Waiting at a junction has no timeout. There is no equivalent of `YIELD_LIMIT` for junctions.
- K3** `segment_for_job` ignores each segment's buffer list, so every divert-to-rack job is rejected and re-raised about twice a second.
- K4** The dispatcher grants one permit at a time to the oldest job, so an unservable job starves work for an idle robot on another leg.
- K5** The lay-by is 2.00 m off the aisle where two turning robots need 2.14 m, and it lands 0.10 m from the parking row.